Robotik Mühendisliği Serisi — Kitapçık 5

Algı, Kestirim ve SLAM

Cilt 5'in telefonda okunacak sürümü

Kodlar bu sürümde yok (A4 Cilt 5 PDF'inde). Metin, sayısal örnekler, şekiller, 21 soru ve cevapları var.

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 Kamera Modeli ve Kalibrasyon **3**

1.1 İğne deliği modeli 3

1.2 DLT ile kalibrasyon 4

2 Homografi ve RANSAC **6**

2.1 Homografi 6

2.2 RANSAC 6

3 İki Görüntüden 3B: Epipolar Geometri **8**

3.1 Esas matris 8

3.2 8-nokta algoritması ve ayrıştırma 8

4 Nokta Bulutu Hizalama: Kabsch ve ICP **10**

4.1 Kabsch (eşleşmeler biliniyorsa) 10

4.2 ICP (eşleşmeler bilinmiyorsa) 10

5 EKF-SLAM **12**

5.1 Problem 12

6 Poz Grafiği ve Gauss–Newton **14**

6.1 Graf olarak SLAM 14

7 Bacaklı Robot Gövde Durum Kestirimi **16**

8 Gerçek Sistemler Haritası ve SB-1 için Algı **19**

8.1 Bu cilttekiler gerçek sistemlerde nerede? 19

8.2 Modern eğilimler (kavram düzeyinde) 20

8.3 SB-1 için gerçekçi bir algı yolu (bütçesiz) 20

8.4 Kaynaklar 21

9 Sorular **22**

10 Cevaplar **25**

Bu Cilt Hakkında

Bir robot ancak *nerede olduğunu* ve *çevresinin nasıl olduğunu* bildiği kadar iyi kontrol edilebilir. Cilt 5 bunun matematiğini kuruyor: kamera geometrisi (bir piksel dünyada neye karşılık gelir), çok görüntüden 3B, nokta bulutu hizalama, olasılıksal kestirim (EKF-SLAM ve poz grafiği) ve bacaklı robotlara özgü gövde durum kestirimi.

Hiçbir yerde OpenCV, GTSAM, Ceres gibi hazır kütüphane kullanılmadı: her algoritma NumPy veya Eigen ile 30–80 satırda sıfırdan yazıldı. Amaç, ileride bu kütüphaneleri kullanırken "içeride ne döndüğünü" bilmen. Gerçek projede tabii ki kütüphaneyi kullanacaksın (Bölüm 8).

Önkoşullar: Cilt 1 (doğrusal cebir, SVD, dönmeler, en küçük kareler, olasılık), Cilt 4B Bölüm 16–17 (Kalman filtresi). **Staj dönemi için:** telefon sürümü (Kitapçık 5) kodsuz; metin, sayılar, şekiller ve 21 soru-cevap var.

Bölüm	Konu	Kod
1	İğne deliği kamera, bozulma, DLT kalibrasyonu	`p1_camera.py`
2	Homografi, normalizasyon, RANSAC	`p2_homography.py`
3	Epipolar geometri, 8-nokta, üçgenleme, ölçek	`p3_epipolar.py`
4	Kabsch ve ICP (nokta bulutu hizalama)	`p4_icp.py/.cpp`
5	EKF-SLAM ve döngü kapatma	`p5_ekf_slam.py`
6	Poz grafiği ve Gauss–Newton (seyrek)	`p6_posegraph.py/.cpp`
7	Bacaklı robot gövde durum kestirimi	`p7_legged_est.py`, `p7_kf.cpp`
8	Gerçek sistemler haritası ve SB-1 için algı	–
9–10	Sorular ve cevaplar	–

Rastgele sayılar hakkında

Python (NumPy) ve C++ (`std::mt19937`) farklı rastgele sayı üreteçleri kullanıyor; aynı tohum (seed) farklı gürültü verir. Bu yüzden bu ciltte iki dilin sayıları *aynı mertebede* ama basamak basamak aynı değil. Kestirimde tek bir koşu "şanslı" veya "şanssız" olabilir; sonuçları her zaman birkaç tohumla dene.

Bölüm 1

Kamera Modeli ve Kalibrasyon

1.1 İğne deliği modeli

Kamera çerçevesinde (X, Y, Z) noktası, görüntü düzleminde $x = X/Z$, $y = Y/Z$ normalize koordinatlarına, oradan piksele düşer:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ 1 \end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

K iç (intrinsic) parametreler: odak uzaklığı (piksel cinsinden), ana nokta (c_x, c_y), eğiklik $s \approx 0$. $(\mathbf{R}, \mathbf{t})$ **dış** (extrinsic): dünyadan kameraya dönüşüm. "$\sim$" ölçeğe kadar eşitlik: üçüncü bileşene bölünür. **Derinlik kaybolur**: tek bir pikselden noktanın uzaklığını bilemezsin, sadece bir ışın elde edersin.

Lens bozulması. Gerçek lensler düz çizgileri eğer. En basit radyal model: $\mathbf{x}_d = \mathbf{x}(1 + k_1 r^2 + k_2 r^4)$, $r^2 = x^2 + y^2$. $k_1 < 0$ fıçı (geniş açılı kameralar), $k_1 > 0$ yastık. Etki merkezden uzaklaştıkça büyür.

Sayılar: $f = 500$, 640×480

$(0.2,\ -0.1,\ 2.0)$ m noktası: $u = 320 + 500 \cdot 0.1 = 370$, $v = 240 + 500 \cdot (-0.05) = 215$. $k_1 = -0.2$ bozulmasıyla $(369.88,\ 215.06)$: merkeze yakın, etki 0.1 piksel. Köşeye yakın $(0.5,\ 0.4,\ 2.0)$: $(445,\ 340) \to (442.4,\ 338.0)$, yaklaşık 3 piksel. **Çözünürlük:** 2 m'de bir piksel $2/500 = 4$ mm; 5 m'de 10 mm.

1.2 DLT ile kalibrasyon

Konumu bilinen $N \geq 6$ 3B nokta ve görüntüleri varsa, $\mathbf{P} = \mathbf{K}[\mathbf{R}\ \mathbf{t}]$ (3×4, 11 serbestlik) doğrusal olarak bulunur. Her eşleşme, $\mathbf{P}$'nin 12 elemanında doğrusal 2 denklem verir:

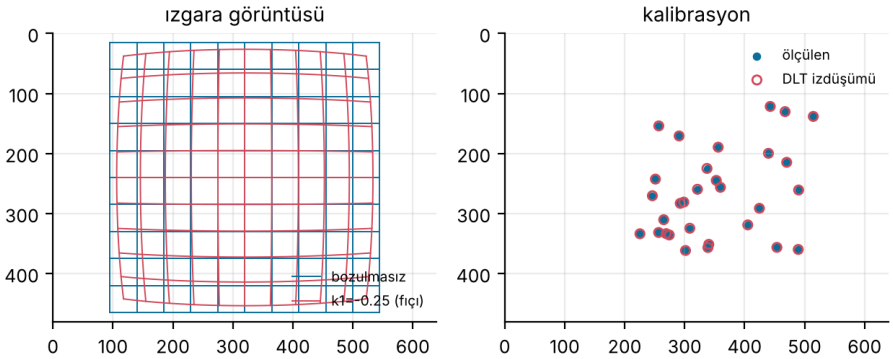
$$\begin{bmatrix} \mathbf{X}^T & \mathbf{0}^T & -u\mathbf{X}^T \\ \mathbf{0}^T & \mathbf{X}^T & -v\mathbf{X}^T \end{bmatrix} \mathrm{vec}(\mathbf{P}) = \mathbf{0}.$$

Hepsini $\mathbf{A}\,\mathrm{vec}(\mathbf{P}) = 0$ olarak yığ; çözüm $\mathbf{A}$'nın en küçük tekil değerine ait sağ tekil vektör (Cilt 1 SVD). Sonra $\mathbf{P}$'nin sol 3×3 bloğu RQ ayrışımıyla $\mathbf{KR}$'ye ayrılır.

Pratikte **Zhang yöntemi** (OpenCV `calibrateCamera`) kullanılır: düz bir dama tahtasının farklı açılardan 10–20 fotoğrafı, her biri için bir homografi (Bölüm 2), ardından tüm parametreler ve bozulma birlikte doğrusal olmayan en küçük karelerle (*yeniden izdüşüm hatası*) iyileştirilir. İyi kalibrasyon: yeniden izdüşüm RMS'i 0.3 pikselin altında.

Sayılar: 30 nokta, 0.5 piksel gürültü

DLT'nin bulduğu $f_x = 496.3$ (gerçek 500), $c = (324.0,\ 247.7)$ (gerçek $(320,\ 240)$), dönme hatası 0.95°, yeniden izdüşüm RMS'i 0.61 piksel. Gürültü büyüdükçe f_x hatası (50 denemenin ortalaması): $\sigma = 0.1 \to 0.7$, $0.5 \to 4.0$, $2 \to 16.4$, $5 \to 44.3$ piksel; kabaca gürültüyle doğrusal.



Şekil 1.1: Sol: düzgün bir ızgaranın bozulmasız (mavi) ve fıçı bozulmalı (kırmızı) görüntüsü. Sağ: kalibrasyon noktaları ve DLT ile yeniden izdüşümü.

Python notu: SVD ile homojen sistem

`_, _, Vt = np.linalg.svd(A); Vt[-1]`: $\|\mathbf{A}\mathbf{x}\|$'i $\|\mathbf{x}\| = 1$ koşuluyla en küçükleyen $\mathbf{x}$. Bu kalıbı bu ciltte dört kez göreceksin (DLT, homografi, 8-nokta, üçgenleme): hepsi aynı "doğrusal denklemleri yığ, SVD'nin son satırını al" fikri.

Bölüm 2

Homografi ve RANSAC

2.1 Homografi

Sahne düzlemselse (zemin, duvar, dama tahtası, AprilTag), iki görüntü arasındaki ilişki bir 3×3 matristir: $\mathbf{x}' \sim \mathbf{H}\mathbf{x}$. Ölçek önemsiz olduğu için 8 serbestlik; her eşleşme 2 denklem: **en az 4 nokta**. Ayrıca kamera yalnızca dönüyorsa (öteleme yok) sahne düzlemsel olmasa da ilişki bir homografidir (panorama). Kullanımlar: AprilTag pozunun bulunması, zemin düzleminin "kuşbakışı" görüntüsü, Zhang kalibrasyonu.

DLT ile çözüm Bölüm 1'deki gibi. **Hartley normalizasyonu:** noktaları merkezleri 0'da, ortalama uzaklıkları $\sqrt{2}$ olacak şekilde ölçekle, çöz, geri dönüştür ($\mathbf{H} = \mathbf{T}_2^{-1}\tilde{\mathbf{H}}\mathbf{T}_1$). Piksel koordinatları (yüzlerce) ile 1'ler aynı matriste karışınca SVD kötü koşullanır.

2.2 RANSAC

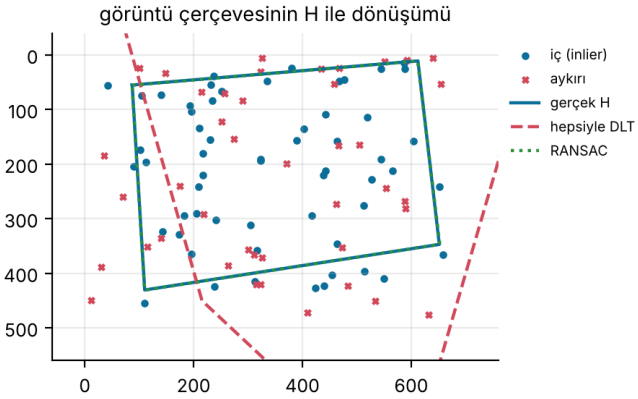
Özellik eşleştirme (ORB, SIFT) eşleşmelerin %20–50'sini yanlış verebilir. En küçük kareler bir tek kötü noktayla bile bozulur. **RANSAC**:

1. Rastgele en küçük örneği seç (homografi için 4).
2. Modeli kestir; hatası eşikten küçük olan noktaları say (iç noktalar, *inliers*).
3. En çok iç noktası olan modeli tut; sonunda tüm iç noktalarla yeniden kestir.

Gereken iterasyon: iç oran w, örnek boyutu s, başarı olasılığı p için $k = \frac{\log(1-p)}{\log(1-w^s)}$.

Sayılar: 100 eşleşme, %40'ı yanlış

Hepsiyle DLT: test noktalarında ortalama hata **460 piksel** (tamamen bozuk). RANSAC (200 iterasyon, 3 piksel eşik): **0.34 piksel**; 60 gerçek iç noktanın 58'ini buldu. Normalizasyonun etkisi bu örnekte küçük (0.35 → 0.27 piksel) ama Bölüm 3'te dramatik.
İterasyon sayısı ($p = 0.99$): $w = 0.9$: 5 ($s = 4$), 9 ($s = 8$). $w = 0.6$: 34 / 272. $w = 0.3$: 567 / **70 188**. Bu yüzden **en küçük örneği** kullanan çözücüler (5-nokta, 3-nokta PnP) değerlidir.



Şekil 2.1: Görüntü çerçevesinin gerçek, "hepsiyle DLT" ve RANSAC homografileri ile dönüşümü. RANSAC gerçek çerçeveyle çakışıyor; aykırı noktalar (x) dışarıda bırakıldı.

Bölüm 3

İki Görüntüden 3B: Epipolar Geometri

3.1 Esas matris

Kamera 1'den kamera 2'ye dönüşüm $\mathbf{X}_2 = \mathbf{R}\mathbf{X}_1 + \mathbf{t}$. Aynı noktanın iki normalize görüntü koordinatı $\mathbf{x}_1, \mathbf{x}_2$ (homojen) için

$$\mathbf{x}_2^T \mathbf{E}\, \mathbf{x}_1 = 0, \qquad \mathbf{E} = [\mathbf{t}]_\times \mathbf{R} \quad \text{(esas matris, } \textit{essential}\text{)}.$$

Geometrik anlamı: $\mathbf{x}_1$'in ışını, ikinci görüntüde bir doğruya (epipolar doğru) düşer; eşi o doğrunun üzerindedir. Piksel koordinatlarıyla aynı denklem $\mathbf{F} = \mathbf{K}^{-T}\mathbf{E}\mathbf{K}^{-1}$ (temel matris, *fundamental*) ile yazılır.

$\mathbf{E}$'nin 5 serbestliği var (dönme 3 + ötelemenin *yönü* 2). **Ölçek kaybı:** $\mathbf{t}$'yi 2 katına, sahneyi de 2 katına çıkarırsan görüntüler aynıdır. Tek kamerayla (monoküler) ölçek *asla* gözlemlenemez; dışarıdan gelmeli: stereo taban çizgisi, IMU, bacak odometrisi, bilinen boyutlu bir nesne.

3.2 8-nokta algoritması ve ayrıştırma

Her eşleşme $\mathbf{E}$'nin 9 elemanında bir doğrusal denklem: $(\mathbf{x}_2 \otimes \mathbf{x}_1)^T \mathrm{vec}(\mathbf{E}) = 0$. 8 eşleşme + SVD. Sonra $\mathbf{E}$'yi tekil değerleri $(1, 1, 0)$ olacak şekilde düzelt. $\mathbf{E} = \mathbf{U}\mathrm{diag}(1,1,0)\mathbf{V}^T$'den dört aday çıkar: $\mathbf{R} \in \{\mathbf{U}\mathbf{W}\mathbf{V}^T, \mathbf{U}\mathbf{W}^T\mathbf{V}^T\}$, $\mathbf{t} = \pm\mathbf{u}_3$. Doğrusu, noktaları *iki kameranın da önüne* düşüren tektir (*cheirality*).

Üçgenleme: iki ışının "kesişimi". $\mathbf{P}_1 = [\mathbf{I}\ \mathbf{0}]$, $\mathbf{P}_2 = [\mathbf{R}\ \mathbf{t}]$ için her görüntü 2 doğrusal denklem verir; 4 denklem, SVD.

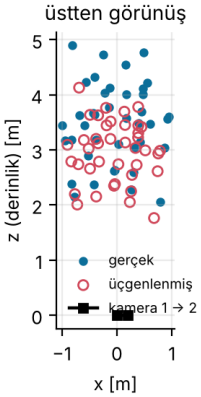
Derinlik belirsizliği (bilinen poz, taban çizgisi b): $z = fb/d$ (d: piksel farkı, *disparity*) $\Rightarrow$ $\sigma_z \approx \frac{z^2}{fb}\sigma_d$. Derinlik hatası uzaklığın *karesiyle* büyür.

Sayılar: 40 nokta, 2–5 m, 20 cm yan hareket

Öteleme yönü hatası (30 deneme medyanı):

gürültü	ham 8-nokta	Hartley normalizasyonlu
0.1 px	1.0°	1.5°
0.5 px	39.4°	6.4°
1.0 px	**84.7°**	6.6°

Normalizasyon burada hayati. Seçilen çözüm: dönme hatası 0.56°, öteleme yönü 6.2°; $|\mathbf{t}|$ tahmini 1 (ölçek yok!). Ölçeği dışarıdan verince 3B noktaların medyan hatası 0.47 m; derinliğe göre 0.23 / 0.46 / 0.71 m (2–3 / 3–4 / 4–5 m). **Poz biliniyorsa** 3.5–4.5 m'de derinlik hatası 0.068 m; formül $16 \cdot 0.5/(500 \cdot 0.2) = 0.08$ m. Fark poz kestirimindeki hatadan geliyor: bu yüzden gerçek sistemler poz ve noktaları *birlikte* iyileştirir (**bundle adjustment**, Bölüm 6'daki Gauss–Newton'un aynısı).



Şekil 3.1: Üstten görünüş: gerçek noktalar ve üçgenlenmiş noktalar. Uzaktaki noktalar daha hatalı (derinlik hatası $\propto z^2$).

Bölüm 4

Nokta Bulutu Hizalama: Kabsch ve ICP

4.1 Kabsch (eşleşmeler biliniyorsa)

$\mathbf{q}_i \approx \mathbf{R}\mathbf{p}_i + \mathbf{t}$ en küçük kareler problemi kapalı formda çözülür:

1. Ağırlık merkezlerini çıkar: $\mathbf{p}'_i = \mathbf{p}_i - \bar{\mathbf{p}}$, $\mathbf{q}'_i = \mathbf{q}_i - \bar{\mathbf{q}}$.
2. $\mathbf{H} = \sum \mathbf{p}'_i \mathbf{q}'^T_i$, SVD: $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
3. $\mathbf{R} = \mathbf{V}\,\mathrm{diag}(1, \dots, 1, \det(\mathbf{V}\mathbf{U}^T))\,\mathbf{U}^T$ (son eleman yansımayı engeller), $\mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}}$.

Aynı formül 2B ve 3B'de çalışır; AprilTag köşelerinden, stereo noktalarından veya hareket yakalama işaretlerinden poz bulmanın çekirdeğidir.

4.2 ICP (eşleşmeler bilinmiyorsa)

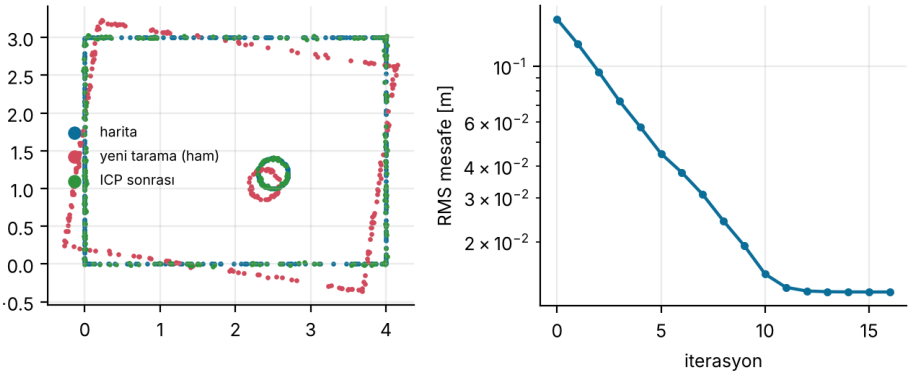
Lidar veya derinlik kamerası iki taramada *aynı* noktaları ölçmez. **Iterative Closest Point:** (1) mevcut tahminle taramayı dönüştür, (2) her noktaya haritadaki en yakın noktayı eşle, (3) uzak eşleşmeleri at, (4) Kabsch ile adımı çöz, (5) tekrarla. Yerel bir yöntemdir: **iyi bir başlangıç tahmini gerekir** (odometri, IMU). Varyantlar: nokta-düzlem ICP (daha hızlı yakınsar), GICP, NDT; en yakın komşu için k-d ağacı.

Sayılar: 4×3 m oda + kolon, 1 cm gürültü

Gerçek hareket 8.59°, (0.30, −0.20) m. ICP 17 iterasyonda 8.57°, (0.2997, −0.1998); RMS eşleşme mesafesi 0.154 → 0.013 m (son değer, 1 cm gürültü + tarama seyrekliği). Başlangıç tahmini kötüleşince:

gerçek dönme	8.6°	25.8°	43.0°	65.9°	94.5°
açı hatası	0.07°	0.04°	0.05°	**49.5°**	**89.8°**

≈ 45°'nin ötesinde yerel minimuma düşüyor: dikdörtgen oda 90° simetrisine yakın; duvarlar yanlış duvarlarla eşleşiyor. C++ sürümü de aynı davranışı gösterdi (65.9°'de 13.5°'de takıldı).



Şekil 4.1: Sol: harita, ham yeni tarama ve ICP sonrası. Sağ: RMS eşleşme mesafesinin iterasyonlarla düşüşü.

C++ notu: Eigen'de satır işlemleri

`M.rowwise() - Sw.row(i)` her satırdan bir satır vektörünü çıkarır (NumPy'daki yayınlamanın, *broadcasting*, karşılığı); `.rowwise().squaredNorm().minCoeff(&j)` en küçük değeri ve indisini birlikte verir. `using Cloud = Eigen::Matrix<double, Eigen::Dynamic, 2>`: satır sayısı çalışma zamanında, sütun sayısı derleme zamanında belli; bu, boyut hatalarını derleyicinin yakalamasını sağlar.

Bölüm 5

EKF-SLAM

5.1 Problem

SLAM (eşzamanlı konum belirleme ve haritalama): robot hem kendi pozunu hem de işaretlerin (landmark) konumlarını, ikisini de bilmeden kestirir. Bu bir tavuk-yumurta problemi gibi görünür, ama olasılıksal olarak tek bir durum vektörüyle çözülür:

$$\mathbf{x} = [x,\ y,\ \theta,\ l_{1x},\ l_{1y},\ \ldots,\ l_{nx},\ l_{ny}]^T.$$

Tahmin: sadece robot kısmı hareket modeliyle ilerler; kovaryansın robot satırları $\mathbf{F}_x$ ile çarpılır. **Güncelleme:** işaret i'ye uzaklık-yön ölçümü $\mathbf{z} = (r,\ \phi)$; Jacobian $\mathbf{H}$ sadece robot ve i. işaret sütunlarında sıfırdan farklı. Ama Kalman kazancı *tüm* durumu günceller: bir işareti görmek, onunla *ilişkili* (korelasyonlu) tüm işaretlerin ve robotun tahminini düzeltir. SLAM'in sihri bu korelasyonlardadır.

Döngü kapatma: robot uzun süre yeni bölgelerde gezip belirsizliği büyüdükten sonra eski bir işareti tekrar görünce, belirsizlik bir anda çöker ve yol boyunca biriken hata geriye doğru düzelir.

EKF-SLAM'in sınırları

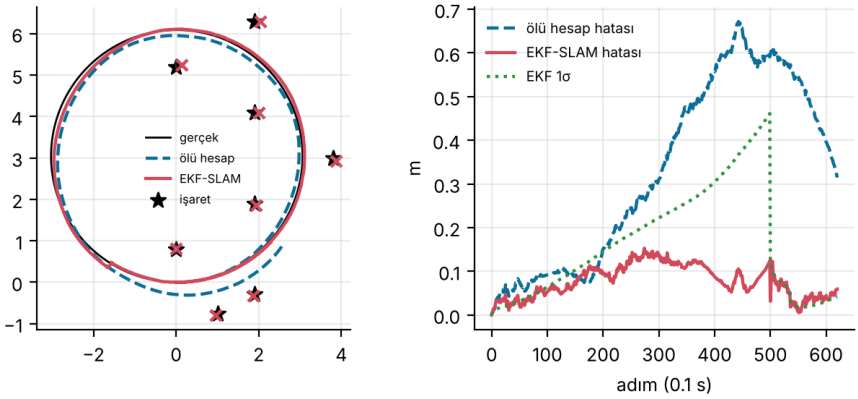
Kovaryans $(3+2n)^2$: 1000 işarette 4 milyon eleman, her güncelleme $O(n^2)$. Doğrusallaştırma hataları zamanla filtreyi *aşırı emin* (tutarsız) yapabilir. Yanlış eşleştirme (data association) tek başına haritayı bozar. Bu yüzden modern sistemler poz grafiği (Bölüm 6) veya kayan pencere optimizasyonu kullanır. EKF-SLAM yine de kavramları öğrenmek için en temiz yoldur.

Sayılar: ∼3 m yarıçaplı daire, işaretler sadece yarısında, menzil 1.6 m

Robot 321.–499. adımlar arasında hiçbir işaret görmüyor.

adım	ölü hesap hatası	EKF-SLAM hatası	EKF 1σ
100	0.092	0.054	0.064
300	0.310	0.135	0.222
400 (kör)	0.557	0.088	0.305
470 (kör)	0.587	0.067	**0.409**
520 (döngü kapandı)	0.579	0.080	**0.073**
600	0.396	0.045	0.032

Kör bölgede filtre belirsizliğinin büyüdüğünü dürüstçe söylüyor (1σ 0.41 m), işaretleri tekrar görünce 0.07 m'ye çöküyor. Bu koşuda kör bölgedeki gerçek hata (0.07 m) şanslı çıktı; önemli olan 1σ'nın davranışı. 8 işaretin hataları 0.6–15 cm. Son adımda NEES 3.85 (2 serbestlik için beklenen ortalama 2; tek değer bunun etrafında dağılır, tutarlılığı ancak birçok koşunun ortalaması söyler).



Şekil 5.1: Sol: gerçek yörünge, ölü hesap ve EKF-SLAM; yıldızlar gerçek, çarpılar kestirilen işaretler. Sağ: konum hataları ve EKF'nin kendi belirsizliği; döngü kapanınca belirsizlik çöküyor.

Bölüm 6

Poz Grafiği ve Gauss–Newton

6.1 Graf olarak SLAM

Filtre yerine *tüm geçmişi* bir optimizasyon problemi olarak kur: düğümler pozlar $\mathbf{x}_i = (x_i, y_i, \theta_i)$; kenarlar göreli ölçümler $\mathbf{z}_{ij}$ (odometri veya döngü kapatma) ve bilgi matrisleri $\mathbf{\Omega}_{ij} = \mathbf{\Sigma}_{ij}^{-1}$:

$$\min_{\mathbf{x}} \sum_{(i,j)} \mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}, \qquad \mathbf{e}_{ij} = \mathrm{t2v}\big(\mathbf{Z}_{ij}^{-1}\, \mathbf{X}_i^{-1} \mathbf{X}_j\big)\,.$$

Gauss–Newton: hatayı doğrusallaştır, $\mathbf{e}_{ij}(\mathbf{x} + \Delta\mathbf{x}) \approx \mathbf{e}_{ij} + \mathbf{A}_{ij}\Delta\mathbf{x}_i + \mathbf{B}_{ij}\Delta\mathbf{x}_j$; normal denklemler $\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$, $\mathbf{H} = \sum \mathbf{J}^T\mathbf{\Omega}\mathbf{J}$, $\mathbf{b} = \sum \mathbf{J}^T\mathbf{\Omega}\mathbf{e}$. Her kenar $\mathbf{H}$'ye sadece dört 3×3 blok ekler: $\mathbf{H}$ **seyrek**tir. İlk pozu sabitle (aksi halde tüm grafiği kaydırmak maliyeti değiştirmez: *gauge* özgürlüğü, $\mathbf{H}$ tekil).

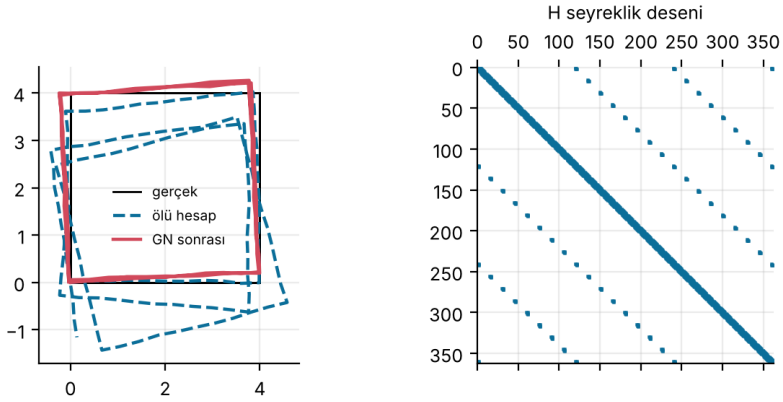
Sağlık kontrolü: gürültü modeli doğruysa optimumda $\chi^2 \approx$ (ölçüm sayısı − bilinmeyen sayısı). Çok büyükse ya model yanlış ya da yanlış bir döngü kapatma var (robust kayıp fonksiyonları: Huber, Cauchy).

Bu, **bundle adjustment**'ın (kamera pozları + 3B noktalar, yeniden izdüşüm hatası) ve **faktör grafiklerinin** (GTSAM, Ceres, g2o) aynısıdır; sadece hata fonksiyonları farklı. Cilt 4B'deki trajectory optimization ile de aynı matematik: seyrek en küçük kareler.

Sayılar: 3 tur kare koridor, 121 poz

120 odometri + 27 döngü kapatma kenarı. χ^2: 8666 → 128 → 85.4 → 85.4 (3 iterasyonda yakınsama: Gauss–Newton'un karesel yakınsaması). Beklenen $441-360 = 81$: model tutarlı. Yörünge hatası (ATE RMS) 0.85 → 0.21 m; son poz hatası 1.16 → 0.04 m. $\mathbf{H}$

363×363, sadece %2.4'ü sıfırdan farklı.
C++ (seyrek `SimplicialLDLT`): 3 tur ATE 0.63 → 0.09 m, 2.5 ms; **30 tur (1201 poz, 4710 kenar)** ATE 2.36 → 0.04 m, 7 iterasyon, 105 ms. Yoğun (dense) çözümle 3603×3603 matrisin her iterasyonda ayrıştırılması saniyeler sürerdi.



Şekil 6.1: Sol: ölü hesap (kesik) ve Gauss–Newton sonrası yörünge. Sağ: **H**'nin seyreklik deseni; köşegen bant odometri, dağınık bloklar döngü kapatmalar.

C++ notu: `Triplet` ile seyrek matris kurmak

`std::vector<Eigen::Triplet<double>>` içine (satır, sütun, değer) üçlülerini sırasız ekle; `setFromTriplets` aynı konumdakileri *toplar*. Bu, "her kenar kendi bloklarını ekler" yapısına birebir uyar. `SimplicialLDLT` simetrik pozitif tanımlı seyrek sistemler için Cholesky ayrışımıdır; büyük problemlerde sıralama (AMD) dolgu (fill-in) miktarını belirler.

Bölüm 7

Bacaklı Robot Gövde Durum Kestirimi

Yürüyen bir robotun kontrolcüsü (Cilt 6: MPC, WBC) her adımda gövdenin *hızını* ve *yönelimini* ister. Gövde üzerinde iki iç (proprioceptive) kaynak vardır:

- **IMU:** ivme ve açısal hız, yüksek hızda (200–1000 Hz). Ama ivmeyi iki kez entegre etmek sapmayı (b) $\frac{1}{2}bt^2$ ile büyütür.
- **Bacak kinematiği:** yerdeki ayak kaymıyorsa, ayak hızı sıfırdır: $\dot{\mathbf{p}}_{\text{ayak}} = \mathbf{v}_{\text{gövde}} + \boldsymbol{\omega} \times \mathbf{r} + \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \;\Rightarrow\; \mathbf{v}_{\text{gövde}} = -\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} - \boldsymbol{\omega} \times \mathbf{r}$. Eklem enkoderleri zaten var; ek sensör gerekmez.

Kalman filtresi IMU'yu tahmin adımında, bacak hızını güncelleme adımında kullanır ve ivmeölçer sapmasını da kestirir. Temas, ya kuvvet sensöründen ya da kontrolcünün yürüme fazından ("şu an sol ayak yerde olmalı") bilinir.

Kayma ve kapı: ayak kayarsa "ayak hızı sıfır" varsayımı çöker. Yenilik (innovation) $\nu = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}$ ve kovaryansı $\mathbf{S}$ ile **Mahalanobis uzaklığı** $d^2 = \nu^T \mathbf{S}^{-1} \nu$ hesaplanır; d^2, χ^2 dağılımının yüksek bir yüzdeliğini aşarsa ölçüm reddedilir (1 boyut için $9 = 3\sigma$).

Gözlemlenemezlik: yalnız IMU + bacakla mutlak konum (x, y, z) ve yön açısı (yaw) gözlemlenemez; hata zamanla sınırsız büyür. Yuvarlanma ve yunuslama (roll, pitch) yerçekimi sayesinde gözlemlenebilir. Konumu bağlamak için kamera/lidar (Bölüm 3–6) gerekir.

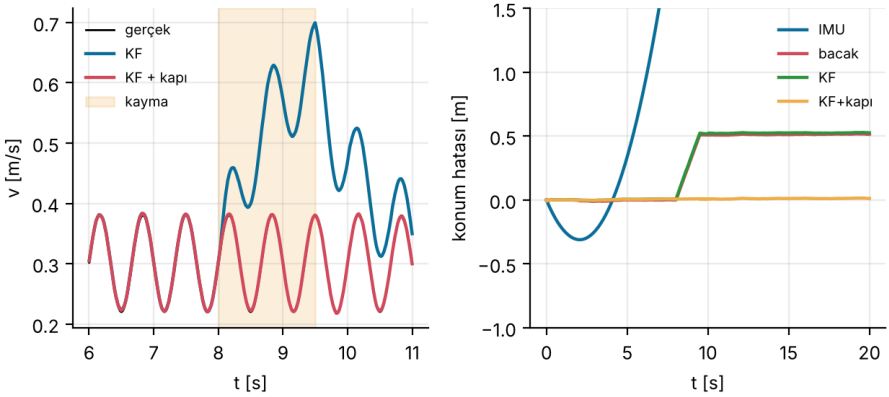
Sayılar: 20 s yürüme, 0.3 m/s, IMU sapması 0.15 m/s^2, 8–9.5 s arası ayak kayması

Yöntem	20 s sonra konum hatası	hız RMS hatası
Sadece IMU (çift integral)	23.8 m	1.47 m/s
Sadece bacak odometrisi	0.51 m	–
KF (IMU + bacak)	0.53 m	0.078 m/s
KF + 3σ kayma kapısı	0.013 m	0.012 m/s

Sapma 0.1494 olarak kestirildi (gerçek 0.15). Kayma sırasında kapısız KF'nin hız hatası 0.21 m/s; kapılı 0.002 m/s (600 ölçümün 583'ü reddedildi). Kapılı KF'nin 1.3 cm'lik konum hatası bu koşuda şanslı; konum gözlemlenemediği için başka tohumlarla birkaç cm'den onlarca cm'ye kadar değişir. C++ şablonlu KF: kapısız 0.53 m / 0.079 m/s, kapılı 0.007 m / 0.012 m/s, 579 ret.

Kapı iki ucu keskin bir bıçak

Kestirim bir kez gerçekten saparsa, doğru ölçümler de "tutarsız" görünüp sürekli reddedilebilir ve filtre bir daha toparlanmaz. Gerçek sistemlerde art arda çok ret olursa kapı gevşetilir veya filtre yeniden başlatılır.



Şekil 7.1: Sol: kayma aralığında (sarı) gövde hızı; kapısız KF kayan ayağa inanıyor. Sağ: konum hataları; IMU tek başına dakikalar içinde işe yaramaz hale geliyor.

C++ notu: şablon parametresi olarak boyut

`template <int NX, int NU, int NZ> class KalmanFilter`: durum, giriş ve ölçüm boyutları derleme zamanında sabit. Eigen bu durumda matrisleri yığında (stack) tutar; döngüde bellek ayırma (`malloc`) olmaz. Bu, 1 kHz gerçek zamanlı döngülerde önemlidir (Cilt 2). Kovaryans güncellemesinde **Joseph biçimi** $(\mathbf{I}-\mathbf{KH})\mathbf{P}(\mathbf{I}-\mathbf{KH})^T + \mathbf{KRK}^T$, kısa biçim $(\mathbf{I}-\mathbf{KH})\mathbf{P}$'den daha pahalıdır ama yuvarlama hatalarıyla $\mathbf{P}$'nin simetrikliğini ve pozitifliğini bozmaz.

Bölüm 8

Gerçek Sistemler Haritası ve SB-1 için Algı

8.1 Bu cilttekiler gerçek sistemlerde nerede?

Sistem / araç	Ne yapar	Bu ciltte karşılığı
OpenCV	kalibrasyon, özellikler, homografi, **E**, PnP, RANSAC	Bölüm 1–3
AprilTag	basılı işaret: kimlik + 6B poz	Bölüm 1–2 (homografi), 4 (Kabsch)
ORB-SLAM3	görsel ve görsel-ataletsel SLAM	3 (epipolar), 6 (BA, poz grafiği)
VINS-Mono/Fusion, OpenVINS	görsel-ataletsel odometri	3, 5, 7
Cartographer, KISS-ICP, FAST-LIO2	lidar SLAM / odometri	4 (ICP), 6
GTSAM, Ceres, g2o	faktör grafiği / en küçük kareler çözücüleri	6
`robot_localization` (ROS 2)	EKF/UKF sensör füzyonu	5, 7
Pronto, Cerberus, kontak destekli InEKF	bacaklı robot durum kestirimi	7

8.2 Modern eğilimler (kavram düzeyinde)

- **Öğrenilmiş özellikler ve eşleştiriciler** (SuperPoint, LightGlue gibi): ORB/SIFT'in yerine sinir ağı; geometri (RANSAC, BA) aynı kalır.
- **Değişmez (invariant) EKF:** durumu Lie grubunda tanımlar (Cilt 1 Bölüm 3); bacaklı robotlarda tutarlılığı belirgin biçimde artırır.
- **Derinlik kameraları ve yükseklik haritaları:** bacaklı robotlar için "nereye basabilirim" sorusu (elevation mapping) SLAM kadar önemlidir.
- **Uçtan uca öğrenme:** pekiştirmeli öğrenme ile yürüyen robotlar (Cilt 6) çoğu zaman açık bir harita kurmaz; ama gövde durum kestiricisi yine kullanılır.

8.3 SB-1 için gerçekçi bir algı yolu (bütçesiz)

1. **Telefon kamerası kalibrasyonu:** kâğıda basılmış veya ekranda gösterilen dama tahtası, OpenCV `calibrateCamera`. Yeniden izdüşüm hatasını raporla.
2. **AprilTag pozu:** basılı etiket, `pupil-apriltags` veya OpenCV `aruco`. Etiketi cetvelle hareket ettirip ölçülen mesafeyle karşılaştır (Bölüm 3'teki z^2 yasasını doğrula).
3. **IMU sapması:** elinde bir IMU (MPU6050 sınıfı, ucuz) olursa: 60 s sabit kayıt, Allan varyansı ve Bölüm 7'deki $\frac{1}{2}bt^2$ sapmasını kendin gör.
4. **Simülasyonda SB-1:** MuJoCo'da eklem açıları + gövde IMU'su; Bölüm 7'deki KF'yi gerçek kinematikle ($\mathbf{v} = -\mathbf{J}\dot{\mathbf{q}}$) kur; ayağı kaydırıp kapıyı test et.

8.4 Kaynaklar

- Hartley & Zisserman, *Multiple View Geometry in Computer Vision* (Bölüm 1–3'ün klasik kaynağı).
- Szeliski, *Computer Vision: Algorithms and Applications*, 2. baskı (yazarın sitesinde ücretsiz).
- Barfoot, *State Estimation for Robotics*, 2. baskı (yazarın sitesinde ücretsiz PDF; Lie grupları ile kestirim).
- Thrun, Burgard, Fox, *Probabilistic Robotics* (EKF-SLAM, parçacık filtresi, GraphSLAM).
- Grisetti ve ark., "A Tutorial on Graph-Based SLAM" (Bölüm 6'nın doğrudan kaynağı).
- Bloesch ve ark., "State Estimation for Legged Robots: Consistent Fusion of Leg Kinematics and IMU" (RSS 2012).
- Hartley (R.) ve ark., "Contact-Aided Invariant Extended Kalman Filtering for Robot State Estimation" (IJRR 2020).
- Cyrill Stachniss'in YouTube'daki SLAM ve fotogrametri ders videoları (telefonda izlenebilir, staj dönemi için ideal).

Bölüm 9

Sorular

Kâğıt, kalem, hesap makinesi. **(K)**: kod okuma.

P1 — Kamera

P1.1 $f_x = f_y = 500$, $c = (320,\ 240)$. Kamera çerçevesindeki $(0.2,\ -0.1,\ 2.0)$ m noktasının pikseli? Aynı ışın üzerindeki $(0.4,\ -0.2,\ 4.0)$ noktasınınki?

P1.2 Aynı kamerada 1 piksel, 2 m ve 5 m uzaklıkta kaç milimetreye karşılık gelir? SB-1'in kamerası 1.5 m'deki 2 cm'lik bir engeli kaç piksel genişliğinde görür?

P1.3 $\mathbf{P} = \mathbf{K}[\mathbf{R}\ \mathbf{t}]$ neden 11 serbestliğe sahip (12 eleman olduğu halde)? DLT için en az kaç 3B–2B eşleşme gerekir?

P1.4 $k_1 = -0.2$, normalize koordinat $(0.25,\ 0.2)$. Bozulmuş normalize koordinat? Piksel cinsinden kayma ($f = 500$)?

P2 — Homografi ve RANSAC

P2.1 Homografinin serbestlik sayısı ve gereken en az nokta sayısı? Kamera sadece dönüyorsa neden sahne düzlemsel olmasa da homografi geçerlidir?

P2.2 İç oran %50, örnek boyutu 4 ve 8 için %99 başarıya kaç RANSAC iterasyonu gerekir?

P2.3 Hartley normalizasyonu ne yapar, neden gerekir?

P3 — Epipolar geometri

P3.1 $\mathbf{E}$, $\mathbf{F}$ ve $\mathbf{H}$'nin serbestlik sayıları? $\mathbf{E}$'nin tekil değerleri nasıldır?

P3.2 Tek kamerayla ölçek neden bulunamaz? SB-1'de ölçeği nereden alabilirsin (iki yol)?

P3.3 Stereo kamera: $f = 500$ px, taban $b = 0.1$ m. Piksel farkı 10 px ise derinlik? Piksel farkı ölçüm hatası 0.5 px ise derinlik belirsizliği? Aynı hesap 2 m'de?

P4 — Kabsch ve ICP

P4.1 $\mathbf{p}_1 = (0, 0)$, $\mathbf{p}_2 = (2, 0)$; $\mathbf{q}_1 = (1, 1)$, $\mathbf{q}_2 = (1, 3)$. $\mathbf{R}$ ve $\mathbf{t}$'yi elle bul, sağlamasını yap.

P4.2 ICP neden iyi bir başlangıç tahmini ister? Dikdörtgen bir odada neden $\approx 45°$'den sonra başarısız oldu?

P4.3 (K) `icp` fonksiyonundaki `reject` eşiği ne işe yarar? Çok küçük ve çok büyük seçmenin sonuçları?

P5 — EKF-SLAM

P5.1 2B EKF-SLAM'de 50 işaret: durum boyutu ve kovaryanstaki eleman sayısı?

P5.2 Tek boyutlu KF: önsel $\hat{x} = 10$, $P = 4$; ölçüm $z = 12$, $R = 1$. Kalman kazancı, sonsal kestirim ve varyans?

P5.3 Döngü kapatıldığında neden robotun *geçmiş* yörüngesi ve uzaktaki işaretler de düzelir?

P6 — Poz grafiği

P6.1 121 poz, 147 kenar (her biri 3 boyutlu ölçüm), ilk poz sabit. Optimumda beklenen χ^2? Gerçekte 2000 çıksaydı ne düşünürdün?

P6.2 İlk pozu sabitlemezsek $\mathbf{H}$ neden tekil olur?

P7 — Bacaklı kestirim

P7.1 İvmeölçer sapması 0.15 m/s^2, başka hata yok. Çift integral ile 10 s ve 20 s sonra konum hatası? Simülasyonda 20 s sonra neden 30 m değil 23.8 m çıktı?

P7.2 Tek eklemli, 0.3 m'lik dik bacak, ayak yerde sabit, kalça açısal hızı 1 rad/s. Gövdenin yatay hızı?

P7.3 Bacak hızı ölçüm gürültüsü $\sigma = 0.04$ m/s, filtrenin hız varyansı 10^{-4}. Ayak kayıyor ve ölçüm 0.35 m/s sapıyor. Mahalanobis d^2? 3σ kapısı ($d^2 > 9$) ölçümü reddeder mi?

Bölüm 10

Cevaplar

P1

P1.1 $u = 320 + 500 \cdot 0.2/2 = 370$, $v = 240 + 500 \cdot (-0.1)/2 = 215$. İkinci nokta aynı ışında: yine $(370,\ 215)$. Derinlik görüntüde kaybolur.

P1.2 $2/500 = 4$ mm, $5/500 = 10$ mm. 1.5 m'de piksel 3 mm: $20/3 \approx 6.7$ piksel.

P1.3 Homojen: $\mathbf{P}$ ve $\lambda\mathbf{P}$ aynı izdüşümü verir, ölçek serbest $\Rightarrow 12-1 = 11$. Her eşleşme 2 denklem: en az 6 (5.5'in yukarı yuvarlanması).

P1.4 $r^2 = 0.0625 + 0.04 = 0.1025$; çarpan $1 - 0.2 \cdot 0.1025 = 0.9795$; $(0.2449,\ 0.1959)$. Kayma $500 \cdot \sqrt{0.0051^2 + 0.0041^2} \approx 3.3$ piksel.

P2

P2.1 8 serbestlik, 4 nokta. Saf dönmede $\mathbf{x}_2 \sim \mathbf{K}\mathbf{R}\mathbf{K}^{-1}\mathbf{x}_1$: derinlikten bağımsız, yani bir homografi.

P2.2 $k = \log 0.01 / \log(1-0.5^4) = 72$; $s = 8$: $\log 0.01 / \log(1-0.5^8) = 1177$.

P2.3 Noktaları merkezi sıfır, ortalama uzaklığı $\sqrt{2}$ olacak şekilde öteler ve ölçekler. Ham piksellerde $\mathbf{A}$ matrisinde 10^0 ile 10^5 mertebesi elemanlar karışır, SVD kötü koşullanır; gürültü sonucu büyütülerek bozar (Bölüm 3: 84.7° yerine 6.6°).

P3

P3.1 **E**: 5, **F**: 7, **H**: 8. **E**'nin tekil değerleri $(\sigma, \sigma, 0)$.

P3.2 $\mathbf{t}$ ve sahne birlikte ölçeklenince görüntüler değişmez. Ölçek: bacak odometrisi (adım uzunluğu kinematikten bilinir), IMU (ivme metre cinsinden), bilinen boyutlu bir işaret (AprilTag kenar uzunluğu), stereo.

P3.3 $z = fb/d = 500 \cdot 0.1/10 = 5$ m; $\sigma_z = z^2\sigma_d/(fb) = 25 \cdot 0.5/50 = 0.25$ m. 2 m'de: $4 \cdot 0.5/50 = 0.04$ m.

P4

P4.1 $\bar{\mathbf{p}} = (1, 0)$, $\bar{\mathbf{q}} = (1, 2)$; merkezlenmiş: $\mathbf{p}' = (\mp 1, 0)$, $\mathbf{q}' = (0, \mp 1)$. x ekseni y eksenine gidiyor: $\mathbf{R} = \mathrm{Rot}(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. $\mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}} = (1, 2) - (0, 1) = (1, 1)$. Sağlama: $\mathbf{R}(2, 0) + \mathbf{t} = (0, 2) + (1, 1) = (1, 3)$.

P4.2 En yakın komşu eşleşmeleri ancak tahmin gerçeğe yakınsa doğrudur; ICP yerel bir iniş yöntemi. Dikdörtgen oda 90° döndürülünce kendine çok benzer (kolon dışında); 45°'nin ötesinde duvarlar yanlış duvarlara daha yakın kalır ve ICP o yerel minimuma iner.

P4.3 Haritada karşılığı olmayan (yeni görülen, örtülü) noktaların yanlış eşleşmelerini atar. Çok küçük: başta doğru eşleşmeler bile atılır, yakınsama başlamaz. Çok büyük: aykırılar Kabsch'ı çeker, sonuç kayar.

P5

P5.1 $3 + 2 \cdot 50 = 103$; $103^2 = 10\,609$ eleman.

P5.2 $K = P/(P+R) = 0.8$; $\hat{x}^+ = 10 + 0.8 \cdot 2 = 11.6$; $P^+ = (1-0.8) \cdot 4 = 0.8$.

P5.3 Robotun pozu ile gördüğü her işaret arasında kovaryans terimleri birikir. Eski işareti görmek robotu düzeltir; Kalman kazancı bu

korelasyonlar üzerinden düzeltmeyi bağlı tüm işaretlere yayar. (EKF geçmiş pozları tutmaz ama onlarla ilişkili işaretleri düzeltir; poz grafiği geçmiş pozları da açıkça düzeltir.)

P6

P6.1 $3 \cdot 147 - 3 \cdot 120 = 81$. 2000: ya gürültü modelleri çok iyimser ($\mathbf{\Omega}$ fazla büyük) ya da yanlış bir döngü kapatma var; tek tek kenar hatalarına bak, robust kayıp kullan.

P6.2 Tüm pozları birlikte aynı miktarda kaydırmak/döndürmek göreli ölçümlerin hiçbirini değiştirmez; maliyet o yönde sabit $\Rightarrow$ $\mathbf{H}$'nin 3 boyutlu sıfır uzayı var.

P7

P7.1 $\frac{1}{2} \cdot 0.15 \cdot 10^2 = 7.5$ m; 20 s: 30 m. Simülasyonda filtre hızı 0'dan başlattı ama gerçek hız 0.3 m/s idi: bu ters yönde $0.3 \cdot 20 = 6$ m hata ekler; $30 - 6 \approx 24$ m.

P7.2 Ayak sabit, bacak tepesi (kalça) ayağın etrafında döner: $v = \ell\dot{q} = 0.3 \cdot 1 = 0.3$ m/s (yön dönme yönüne göre).

P7.3 $S = 0.04^2 + 10^{-4} = 0.0017$; $d^2 = 0.35^2/0.0017 = 72$. $72 > 9$: reddedilir.